

TRABAJO ENTREGABLE PATRONES DE COMPORTAMIENTO OBSERVER PATTERN

ÍNDICE

ÍNDICE.....	2
1. ORGANIZACIÓN DE LA ACTIVIDAD PRÁCTICA	3
2. DESCRIPCIÓN DE LA ACTIVIDAD PRÁCTICA.....	3

1. ORGANIZACIÓN DE LA ACTIVIDAD PRÁCTICA

Nombre de la tarea	"Caso de uso del patrón observador"
Tipo de tarea	Individual
Entregables	ObserverPattern_NombreApellidos.pdf

2. DESCRIPCIÓN DE LA ACTIVIDAD PRÁCTICA

En un único documento, formato pdf, se espera la entrega del código fuente junto con el diagrama UML completo que represente la solución al problema propuesto, una solución **que utiliza el patrón de comportamiento visto en la unidad 3, patrón observador (Observer pattern)**. No olvides el uso de la notación camelCase para la escritura del código, también incluye pequeños comentarios en el código que indiquen los componentes principales de los patrones. El cuerpo de los métodos que realizan acciones tan sólo debe lanzar un mensaje por consola indicando la acción invocada.

El alcance de esta solución corresponde al **patrón Observer** con el objeto de **difundir** la información del hecho **"se ha devuelto un libro en mal estado en la biblioteca"**. Siguiendo el diagrama UML de partida, el modelo observable tiene la capacidad de notificar bajo protocolo Pull y/o Push, debido a esta capacidad de emitir la notificación, para los observadores se aprecia que cada uno de los observadores puede consumir tanto el modelo observable como el objeto correspondiente con la variable de estado.

Debido a esta posibilidad de consumir la información por ambos protocolos, tanto por protocolo Pull como Push, se establece una **prioridad de consumo**, de tal manera que si el observador recibe la variable de estado lo consumirá directamente, sólo si no éste no estuviera informado consultará la variable de estado a través del modelo observable.

Además, debido a que la interfaz del observador es genérica, antes de consumir la información el observador deberá asegurarse de que efectivamente lo que recibe es la variable de estado o modelo observable que espera consumir.

EJERCICIO

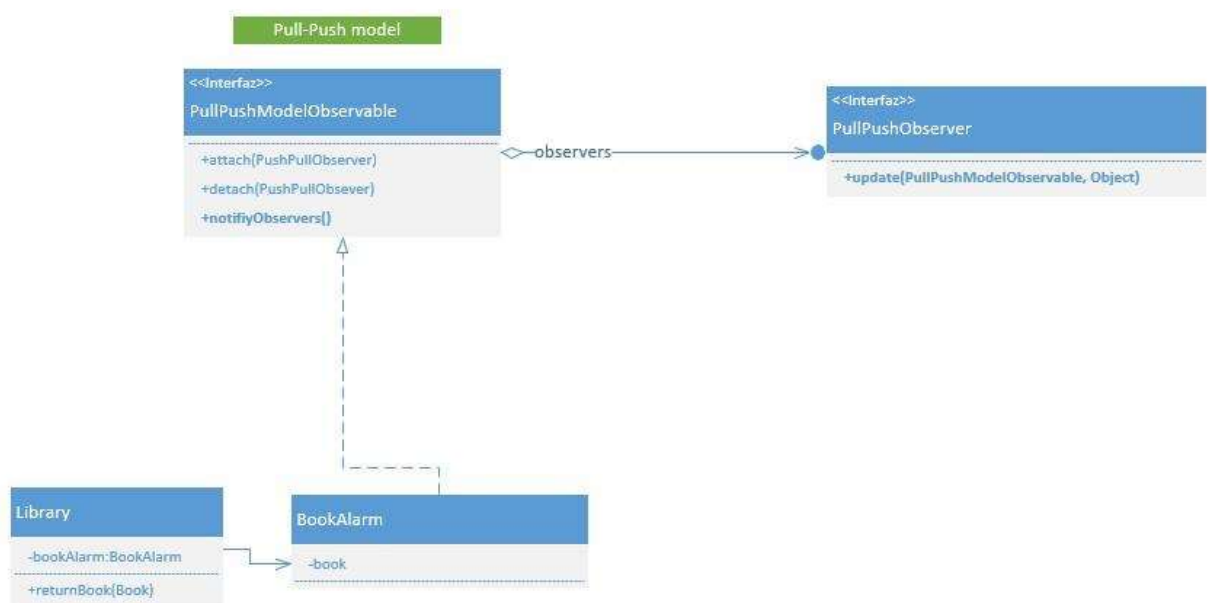
Tomaremos como ejemplo la dinámica de devolución de libros en una biblioteca. El propósito de este caso de estudio es desencadenar los procesos necesarios en caso de que el libro sea devuelto en mal estado.

Se instrumenta un mecanismo para notificar a las áreas interesadas (Stock, Compras, Administración) cada vez que un libro se devuelve en mal estado. El proceso comienza cada vez que un lector devuelve un libro con la ejecución del método **devuelveLibro()** que pertenece a la clase Biblioteca.

Si el lector devolvió el libro dañado, entonces la aplicación avisa a las clases Stock, Compras y Administración, que se ven afectadas por este hecho. Cada vez que el libro se devuelve en mal estado avisa a un objeto de tipo **AlarmaLibro**; esta clase es el sujeto de interés para Stock, Compras y Administración. Dichas clases se convierten en observadoras de la clase **AlarmaLibro**, que tiene la responsabilidad de notificarles el mal estado del libro (el libro de que se trata). Para poder recibir esta notificación las clases deben suscribirse como observadoras, suscripción de la cual pueden desagregarse posteriormente.

La biblioteca tiene un modelo observable (alarma de libro en mal estado) con dos formas posibles de protocolo de comunicación (push y pull), los observadores deben utilizar una interface de observador que permita ambos protocolos.

Este es el diseño inicial de partida.



Se pide:

- **Dibujar un nuevo diagrama UML con los cambios necesarios en el diseño inicial dado para que, mediante la aplicación del patrón Observador, los departamentos Stock, Compras y Administración sean observadores exclusivamente de los libros en mal estado.**
- **Indicar los componentes que pertenecen al patrón observador en el diseño final.**
Aquí se espera que se indique qué clase hace de modelo observable, cuál es la variable de estado del modelo observable, qué clase tiene el rol de observador, etc.
- Provocar en el código que el modelo observable notifique por protocolo pull-push
- Asegurarse en el código de los observadores que tienen prioridad de consumir el cambio de estado por protocolo Push, pero pueden acceder al cambio de estado también por protocolo Pull. También, hay que asegurarse que el modelo observable es del tipo al que se han suscrito.
- Dada esta clase Test, escribe las líneas de código correspondientes para que se cumpla lo que se pide en cada comentario TODO

```
public class LibraryPullPushObserverTest {
    public static void main(String[] args) {
        Library library = new Library();
        //TODO Instancia los departamentos Stock, Admin y Compras como
        observadores

        //TODO provoca que los departamentos se suscriban a la biblioteca

        Book bookBad = new Book("Programar sin patrones",
                                "desconocido", BookState.BAD);
        Book bookGood = new Book("Gang of four Design patterns",
                                "Erich Gamma, Richard Helm", BookState.GOOD);
        System.out.println("Alarma notifica bajo protocolo PULL-PUSH");
        library.returnBook(bookGood);
        library.returnBook(bookBad);
        //TODO provoca que el departamento de compras borre su suscripción.
        System.out.println("Alarma vuelve a notificar bajo protocolo PULL-PUSH");
        Book otherBookBad = new Book("Programar sin pensar",
                                    "desconocido", BookState.BAD);
        library.returnBook(otherBookBad);

    }
}
```

- **Muestra los resultados que obtienes por consola.** Estos son los resultados que se tienen que obtener por la consola:

Alarma notifica bajo protocolo PULL-PUSH

```
[StockDepartmentObserver Push] se ha devuelto el libro en mal estado
Book [title=Programar sin patrones, author=desconocido, bookState=BAD]
[AdminDepartmentObserver Push] se ha devuelto el libro en mal estado
Book [title=Programar sin patrones, author=desconocido, bookState=BAD]
[ComprasDepartmentObserver Push] se ha devuelto el libro en mal estado
Book [title=Programar sin patrones, author=desconocido, bookState=BAD]
```

Alarma vuelve a notificar bajo protocolo PULL-PUSH

```
[StockDepartmentObserver Push] se ha devuelto el libro en mal estado
Book [title=Programar sin pensar, author=desconocido, bookState=BAD]
[AdminDepartmentObserver Push] se ha devuelto el libro en mal estado
Book [title=Programar sin pensar, author=desconocido, bookState=BAD]
```